

MOVIEMATCH - DEVELOPER'S GUIDE

Table Of Contents

- [1. Project Overview](#)
 - [Tech Stack at a glance:](#)
- [2. Repository Structure](#)
- [3. Getting Started](#)
 - [Prerequisites](#)
 - [Installation](#)
 - [Configuration](#)
 - [Running the Server](#)
 - [Opening the App](#)
 - [Default credentials](#)
- [4. Environment Variables](#)
- [5. Backend References](#)
 - [Server Entry Point](#)
 - [API Routes](#)
 - [Watchlist](#)
 - [TMDB](#)
 - [Mood to Genre ID mapping:](#)
 - [Time to Genre ID mapping:](#)
- [6. Database](#)
- [7. Authentication](#)
- [8. TMBD Integration](#)
 - [Fields used client-side:](#)
- [9. Known Issues](#)

1. Project Overview

MovieMatch is a full-stack web application for discovering movies and TV shows. Users can choose a discovery mode from the following options: mood, time of day, trending, top-rated, or search to get a grid of results pulled directly from TMDB API.

Tech Stack at a glance:

Layer	Technology
Frontend	Vanilla HTML, CSS, JavaScript
Backend	Node.js and Express
Database	SQLite via sql.js (WebAssembly)
Authentication	JWT (jsonwebtoken)
External API	The Movie Database (TMDB)

2. Repository Structure

MovieMatchWebsite/

```
├── client/
│   ├── index.html    # Entry point — auth router only, no UI
│   ├── auth.html     # Login page markup
│   ├── auth.css      # Login page styles (polished, uses Google Fonts)
│   ├── auth.js       # Login form logic
│   ├── app.html      # Main application UI
│   ├── style.css     # Main application styles
│   └── app.js        # Main application logic
└── server/
    ├── server.js     # Entire backend (Express + DB + routes)
    ├── package.json  # Dependencies
    ├── package-lock.json
    └── moviematch.db # SQLite database (auto-created on first run)
```

3. Getting Started

Prerequisites

- [Node.js](#) version 18 or later
- A free TMDB API key, register at: themoviedb.org

Installation

```
# Clone the repo
git clone https://github.com/NgocTrann/MovieMatchWebsite.git
cd MovieMatchWebsite/server

# Install dependencies
npm install
```

Configuration

Create a .env file inside of the `server/` directory

```
TMDB_API_KEY=your_tmdb_api_key_here
# Optional
JWT_SECRET=pick_a_long_random_string
```

Running the Server

```
# Production
npm start

# Development (auto-restarts on file changes, requires nodemon)
npm run dev
```

The server starts on <http://localhost:3000>.

Opening the App

Open client/index.html in a browser directly, or serve the `client/` folder with any static file server. For example:

```
npx serve client
```

Default credentials

Field	Value
-------	-------

Username	admin
Password	abc123

4. Environment Variables

All variables are loaded by `dotenv` from `server/.env` at startup.

Variable	Required	Default	Description
TMDB_API_KEY	Yes	none	API key for The Movie Database. All movie/TV routes fail without this.
JWT_SECRET	No	“moviematch_secret”	Secret used to sign and verify JWT tokens. Use a long random string in production.

5. Backend References

Server Entry Point

`server/server.js` is the entire backend. It handles database setup, authentication, watchlist CRUD, and TMDB proxying. It runs on port 3000.

API Routes

Auth

Method	Route	Auth Required	Description
POST	/api/auth/login	No	Accepts { username, password }, returns { token, user } on success or 401 on failure.

Watchlist

Method	Route	Auth Required	Description
--------	-------	---------------	-------------

GET	/api/watchlist	Yes	Returns all watchlist items sorted by added_at descending.
POST	/api/watchlist	Yes	Adds a title. Body: { tmdb_id, media_type, title, poster, rating }. Returns 200 if it already exists, 201 if newly added.
DELETE	/api/watchlist/:tmdb_id	Yes	Removes the item with the given TMDB ID.

TMDB

Method	Route	Description
GET	/api/trending/:type	This week's trending titles from TMDB.
GET	/api/watchlist	All-time top-rated titles from TMDB.
GET	/api/mood/:type/:mood	Discovers titles by genre, derived from the mood. See mood map below.
GET	/api/time/:type/:time	Discovers titles by genre, derived from time of day. See time map below.
GET	/api/search/:type?q=	Searches TMDB by title. Pass the query string as ?q=.

Mood to Genre ID mapping:

Mood	Genre ID	Genre Name
happy	35	Comedy
sad	18	Drama
scared	27	Horror
romantic	10749	Romance

excited	28	Action
relaxed	35	Comedy
thoughtful	99	Documentary
intense	80	Crime

Time to Genre ID mapping:

Time	Genre ID	Genre Name
day	35	Comedy
night	27	Horror
dawn	10749	Romance
dusk	28	Action

6. Database

MovieMatch uses sql.js, a port of SQLite compiled to WebAssembly. It runs entirely in-process with no external database server. The database is loaded from moviematch.db on startup and written back to disk after every write operation.

<https://sql.js.org/#/>

7. Authentication

Token Details

- Library: jsonwebtoken v9
- Payload: { id: 1, username: "admin" }
- Expiry: 7 days
- Algorithm: HS256 (default)

What Is Not Implemented

- User registration

- Password hashing (bcrypt or similar)
 - Token refresh
 - Token revocation / logout on server side (logout is client-side only — just clears localStorage)
- Per-user data isola

8. TMDB Integration

The server acts as a proxy for all TMDB requests. The API key never reaches the browser.

Base URL: `https://api.themoviedb`

Movie Poster images are constructed client-side:

```
`https://image.tmdb.org/t/p/w500${item.poster_path}`
```

If poster_path is null, a placeholder image from `via.placeholder.com` is used instead.

Fields used client-side:

TMDB Field	Used For
id	Watchlist key (stored as tmdb_id)
title/name	Display title (title for movies, name for TV)
poster_path	Poster image URL
voter_average	Star rating display
release_date / first_air_date	Release Date display

9. Known Issues

Hardcoded credentials: the admin username and password (admin / abc123) are stored as a plain object in `server.js`. Anyone with access to the source code can log in. The password is not hashed.

Single user: there is no user table, no registration, and no per-user data. The watchlist is effectively global.

Duplicate mood mappings: happy and relaxed both map to genre 35 (Comedy) and return identical results.

No loading state: while results are fetching, the results area shows the plain text string "Loading..." with no spinner or visual treatment.